CleanNode: An IoT and Machine Learning Based System for Condition-Driven Monitoring of Public Toilets

1st Adhya Pandey
Dept. of Computer Applications
PES University
Bengaluru, India

2nd Rishika Chowdhary B
Dept. of Computer Applications
PES University
Bengaluru, India

3rd Chinmayi C Gowda
Dept. of Computer Applications
PES University
Bengaluru, India

4th Lavanya R
Dept. of Computer Applications
PES University
Bengaluru, India

5th Dilip Kumar Maripuri
Dept. of Computer Applications
PES University
Bengaluru, India

***Abstract*—Public toilets in Indian cities are usually cleaned on fixed timetables, and their condition is known only when a staff member inspects them. As a result, a facility can remain unusable for hours while a lightly used one is cleaned repeatedly. This paper presents CleanNode, a low-cost system that monitors each toilet cubicle continuously and directs cleaning to where it is needed. Each sensor node uses an ESP32 microcontroller with an MQ-135 gas sensor, a DHT22 temperature and humidity sensor, a PIR occupancy sensor and a YF-S201 water flow sensor, and publishes its readings over MQTT every five seconds. A backend service uses a Random Forest regression model to compute a Predictive Cleanliness Index (PCI) between 0 and 100, applies three-cycle debouncing to suppress false alarms, and detects conditions such as bad odour and flush failure by combining readings from several sensors. Results are stored in a Firebase Realtime Database and presented through a React web dashboard for municipal authorities and an Android application for field staff and the public, through which cleaning duties are assigned and tracked. The system was verified with fifteen manual test cases covering the sensor hardware, the web dashboard and the mobile application, all of which passed.**

***Index Terms*—Internet of Things, smart sanitation, public toilets, ESP32, MQTT, Random Forest, cleanliness index, anomaly detection, smart cities**

I. Introduction

Public toilets in cities such as Bengaluru are maintained on fixed cleaning schedules supported by occasional manual inspection. Under this arrangement the authorities have no direct way of knowing the condition of a facility between visits. Problems such as foul odour from poor ventilation, failure of the water supply or heavy footfall at peak hours often go unnoticed until a user complains, and existing complaint channels are slow to act. Poorly maintained restrooms can also expose users to harmful gases such as ammonia [1], and facilities in poor condition discourage people from using them, undermining the purpose of public sanitation programmes [2].

Sensor-based monitoring offers an alternative. A low-cost microcontroller with a few sensors can report the condition of a room every few seconds, and such systems have already been demonstrated for restroom hygiene [1], [2], janitor scheduling [3], waste collection [4], [5] and drainage monitoring [6]. However, existing toilet monitoring systems typically concentrate on a single aspect, usually odour or air quality, do not combine readings into an overall assessment of cleanliness, and offer little to the public who use the facilities.

This paper presents CleanNode, a system that places a sensor node in each cubicle, combines its readings into a single Predictive Cleanliness Index (PCI), and links every alert to the assignment of a cleaning duty. The main contributions are:

- a multi-sensor node and cloud pipeline that reports gas level, temperature, humidity, occupancy and water flow for every cubicle in real time;
- a cleanliness index computed by a Random Forest regression model, with three-cycle debouncing and multi-sensor anomaly detection, including detection of flush failure by relating occupancy to water flow;
- a web dashboard and an Android application that connect alerts to duty assignment for authorities and staff, and give the public access to toilet locations and feedback.

The rest of the paper is organised as follows. Section II reviews related work. Section III describes the system architecture, and Section IV the design of the sensor node, the data model and the cleanliness and alerting logic. Section V presents the implementation, Section VI the testing and results, and Section VII concludes the paper.

II. Related Work

Deshmukh *et al.* [2] proposed an IoT-based public toilet monitoring system using a NodeMCU ESP8266 with sensors for odour, usage, water level, soap level and lighting, RFID tracking of cleaners and on-site rating buttons. Data was sent to Firebase every five minutes, and an ARIMA model forecast

daily trends. The system did not combine its readings into a single cleanliness score.

Azman *et al.* [1] monitored ammonia with an MQ-137 sensor and indoor air quality (IAQ) with a BME680, sending Telegram alerts when ammonia exceeded 5 ppm or IAQ exceeded 100, and triggering cleaning after every 50 users. They found a strong negative correlation ($r = -0.91$) between temperature and humidity and ammonia level. The system assesses air quality only and cannot detect dirty surfaces or flush failure.

Lokman *et al.* [3] combined an ARIMA model for predictive maintenance with a genetic algorithm for janitor scheduling, reducing cleaning staff requirements by up to 24.7% in simulations based on data from university toilets. The work was evaluated in a single building.

Several studies from related domains inform the design of CleanNode. ProWaste [4] used machine learning on fifteen features to predict the criticality of waste collection centres and alerted crews through a mobile application. Ha *et al.* [7] fused several air quality measurements into a single index using an extended fractional-order Kalman filter, showing that a fused index is more reliable than individual sensors. Vibhute *et al.* [6] used a YF-S201 flow sensor to detect drainage blockages from reduced flow, an idea reused here for flush monitoring. Zhang *et al.* [8] achieved alert latency below 450 ms in a public safety system combining edge and cloud processing, and Dineva and Atanasova [9] showed that a cloud architecture can scale automatically to handle data from 10,000 devices per second. Pardini *et al.* [5] demonstrated the value of sharing sensor data with citizens, and Romano *et al.* [10] found that gamification raised participation in a civic application.

From this review, the gaps addressed by CleanNode are the absence of flush health monitoring, of a single combined cleanliness score, of usage-driven cleaning and of a public-facing interface in existing toilet monitoring systems.

III. System Architecture

CleanNode is organised into four layers, shown in Fig. 1. The *IoT layer* consists of a sensor node in each cubicle. An ESP32 microcontroller reads an MQ-135 gas sensor for air quality, a DHT22 for temperature and humidity, a PIR sensor for occupancy and a YF-S201 flow sensor for water usage, and publishes the readings over Wi-Fi using MQTT.

The *database layer* is a Firebase Realtime Database, which holds the live readings together with users, alerts, staff data, reports and the cleanliness thresholds. The *application layer* contains the MQTT broker that receives sensor messages, a REST API that serves the user interfaces, the AI/ML module that computes the PCI and detects anomalies, and a cloud sync service that pushes updates in real time. The *user interface layer* provides a React web dashboard for the municipal authority and an Android application, built with Kotlin, for staff in the field and members of the public.

Four types of user interact with the system. The municipal authority monitors facilities and views analytics and reports; maintenance staff receive alerts and report the completion of cleaning tasks; a supervisor manages staff and assigns tasks; and members of the public view the availability and cleanliness of toilets and submit feedback.

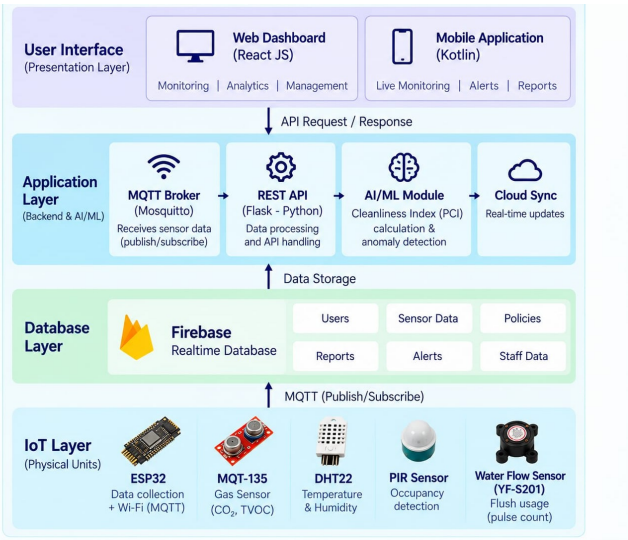


Fig. 1. Layered architecture of CleanNode.

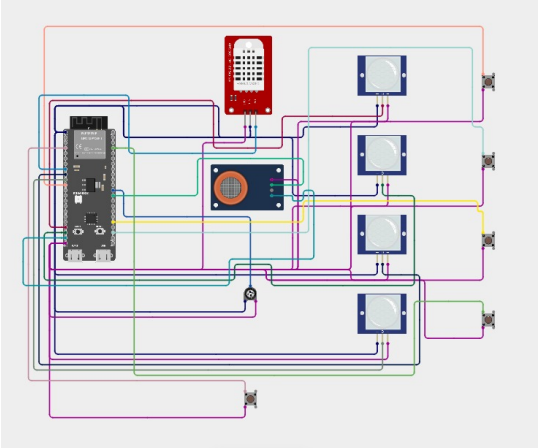


Fig. 2. Sensor integration with the ESP32 microcontroller.

IV. System Design

A. Sensor Node

Fig. 2 shows the wiring of the sensor node. The ESP32 acts as the central controller and collects data from the gas, temperature and humidity, occupancy and water flow sensors. The DHT22 is sampled every 2 s. The YF-S201 produces pulses whose frequency F (Hz) is related to the flow rate Q (L/min) by

$$F = 7.5\,Q. \tag{1}$$

Every 5 s the node builds a JSON message containing the latest readings and publishes it to the MQTT topic `pes/cleannode/iot`. A HiveMQ Cloud broker relays the messages to the backend.

Algorithm 1: PCI computation and alert generation

for each message received from the broker **do**
  extract g, h, T, o, w
  $\mathrm{PCI} \leftarrow f_{\mathrm{RF}}(g, h, T, o, w)$
  store reading and PCI in Firebase
  $c \leftarrow$ condition: Bad Smell, Flush Issue, Danger Zone or Normal
  if $c \neq$ Normal **and** c found in 3 consecutive readings **then**
    write alert to `alerts`; start 15-minute timer
  end if
  if a critical alert is unresolved when its timer expires **then**
    escalate the alert on the dashboard
  end if
end for

B. Data Model

Being document oriented, the Firebase Realtime Database stores data as a tree of JSON nodes. Its seven top-level nodes are `facilities`, holding live readings nested by facility, block and cubicle; `alerts`, holding active Flush Issue, Bad Smell and Danger Zone alerts; `assignments`, recording cleaning duties; `history`, a time series of readings; `processedData`, holding computed PCI values; `staff`, holding staff profiles; and `users`, mapping each Firebase Authentication account to the role Admin, Staff or Public. Each reading is stored as a timestamped document with the fields `gas`, `humidity`, `occupancy`, `water_flow`, `pci`, `timestamp` and `alert`.

C. Cleanliness Index and Alert Generation

For each message received, the backend extracts the gas level g, humidity h, temperature T, occupancy o and water flow w and passes them to a trained Random Forest regression model, which returns the cleanliness index

$$\mathrm{PCI} = f_{\mathrm{RF}}(g, h, T, o, w), \qquad 0 \leq \mathrm{PCI} \leq 100. \tag{2}$$

The reading and its PCI are written to the database, and the reading is then checked against three alert conditions: *Bad Smell* when the gas level exceeds a configurable limit, *Flush Issue* when occupancy is detected but no water flow follows, and *Danger Zone* when the PCI falls below a critical limit. To prevent a single noisy reading from raising a false alarm, an alert is raised only when the same condition is found in three consecutive readings. With a 5 s publishing interval, a genuine problem is therefore reported within about 15 s. If a critical alert remains unresolved for 15 minutes, it is escalated on the dashboard and recorded against the facility. Algorithm 1 summarises this procedure.

D. Multi-Sensor Anomaly Detection

In addition to the individual alerts, the system checks each cubicle for combinations of high gas level, abnormal humidity, high occupancy, occupancy without water flow and low PCI. Each cubicle in which such conditions are found together is assigned a severity of Critical, Moderate or Minor according to the number and type of conditions present, and is listed on the dashboard for attention.

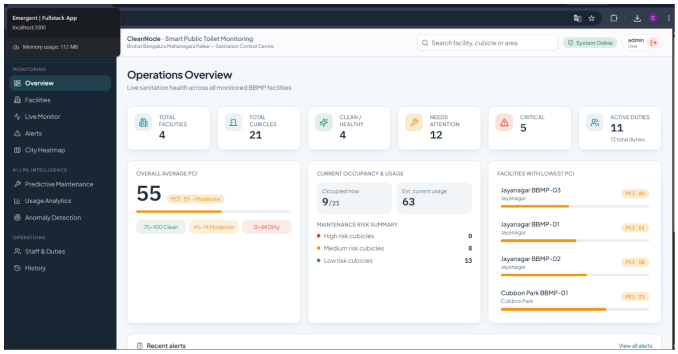


Fig. 3. Operations overview of the web dashboard.

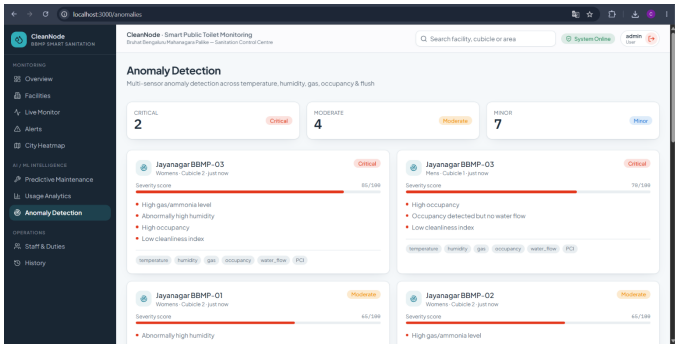


Fig. 4. Anomaly detection page with severity levels.

V. Implementation

The backend is implemented in Python 3.12 using FastAPI, with the Random Forest model built using Scikit-Learn and data processing in Pandas and NumPy. The web dashboard uses React 18, Vite and Tailwind CSS, and the Android application uses Kotlin and Jetpack Compose on Android 14, with Android Room for local storage. User login and roles are managed by Firebase Authentication.

The dashboard provides an operations overview (Fig. 3) showing the total number of facilities and cubicles, how many are clean, need attention or are critical, the number of active duties, the overall PCI and current occupancy. Further pages list each facility with its PCI and alerts, show the latest readings for every cubicle, list detected anomalies with their severity (Fig. 4), and allow the authority to assign a cleaning duty by selecting the facility, cubicle, staff member and priority, or to reassign it if the staff member is unavailable. The Android application shows live key figures, alerts and a map of public toilets, and accepts ratings and comments from the public.

TABLE I
SUMMARY OF TEST CASES AND OBSERVED RESULTS

ID	Module	Test condition	Observed result	Status
TC-HW-01	Gas sensor (MQ-135)	Exposure to test smoke / foul air	Reading rose from 420 ppm to 2850 ppm, above the 2500 ppm threshold	Pass
TC-HW-02	Motion sensor (PIR)	Person moving in front of sensor	Occupancy status changed to Occupied immediately	Pass
TC-HW-03	Water flow (YF-S201)	Water flow during flush	3.5 L dispensed measured using (1)	Pass
TC-HW-04	DHT22	Continuous sampling every 2 s	26.8 °C and 62% RH recorded consistently	Pass
TC-HW-05	ESP32 data push	280-byte JSON payload every 5 s	Packets delivered to the cloud gateway with no drops	Pass
TC-WEB-01	Admin login	Login through Firebase Authentication	Role verified and login succeeded	Pass
TC-WEB-02	Ward filtering	Switch from ward 153 to ward 111	Data changed instantly without page refresh	Pass
TC-WEB-03	Live update	Change a sensor value in Firebase	New value appeared on the dashboard automatically	Pass
TC-WEB-04	Settings	Change the cleanliness limit	Status updated without restarting the system	Pass
TC-WEB-05	Escalation	Critical issue left unresolved for 15 minutes	Alert shown and the issue recorded	Pass
TC-MOB-01	KPI overview	Launch app connected to Firebase	Live figures shown (5 facilities monitored, average PCI 75/100)	Pass
TC-MOB-02	Spike demonstration	Simulated gas spike	PCI fell into Danger Zone; status turned Critical	Pass
TC-MOB-03	Real-time alerts	Open Alerts after the spike	Alert with timestamp and cubicle ID displayed	Pass
TC-MOB-04	Restroom map	Open the map view	BBMP public toilet locations shown with ward names	Pass
TC-MOB-05	Citizen feedback	Submit a 4/5 rating with comment	Feedback stored in Firebase and confirmed to user	Pass

TABLE II
SAMPLE READINGS STORED BY THE SYSTEM

Condition	Gas	Hum. (%)	Occ.	Flow	PCI
Normal	390	46	1	1	61.3
Bad Smell	494	64	0	0	62.5
Flush Issue	245	46	3	0	33.55

VI. TESTING AND RESULTS

The system was tested manually at three levels: the sensor hardware, the web dashboard and the mobile application. Table I summarises the fifteen test cases and their observed results; all of them passed.

Table II shows three readings stored by the system, one for each condition. In the first, a gas reading of 494 triggers a Bad Smell alert while the PCI remains moderate, since the other readings are normal. In the third, an occupancy count of three without any water flow is recorded as a Flush Issue and the PCI falls to 33.55, showing that the index responds strongly to flush failure.

The tests confirm that each sensor responds to the condition it is meant to detect, that readings reach the cloud without loss at the 5 s interval, that changes in the database appear on the dashboard immediately, and that the alerting and escalation behave as designed. These results are functional; long-term field deployment across many facilities and a quantitative evaluation of the PCI model remain to be carried out.

VII. CONCLUSION AND FUTURE WORK

This paper presented CleanNode, a system that replaces fixed cleaning schedules for public toilets with maintenance driven by the actual condition of each cubicle. Sensor nodes built on the ESP32 report gas level, temperature, humidity, occupancy and water flow over MQTT; a Random Forest model combines these readings into a cleanliness index, with debouncing and multi-sensor anomaly detection; and a web dashboard and Android application link each alert to the assignment of a cleaning duty. All fifteen test cases across the hardware, web dashboard and mobile application passed.

Future work includes solenoid valves that trigger flushing or disinfectant spraying automatically when an ammonia spike is detected; solar panels with LiFePO4 batteries to make each node independent of the power supply; ultrasonic level sensors for water tanks and soap dispensers; diagnostics that detect disconnected or tampered sensors; and geofenced alerts that notify a worker who comes within 100 m of a toilet in the Danger Zone.

ACKNOWLEDGMENT

The authors thank the Department of Computer Applications, PES University, Bengaluru, for supporting this work.

REFERENCES

[1] F. I. Azman, N. L. Saleh, F. Hashim, A. Sali, A. M. Ali, and A. S. M. Noor, "An IoT-based hygiene monitoring system in the restroom," *IEEE Access*, vol. 13, pp. 119348–119361, 2025, doi: 10.1109/ACCESS.2025.3586117.

[2] P. Deshmukh, A. Mohite, H. Bhoir, R. Patil, and A. Bhonde, "Intelligent public toilet monitoring system using IoT," in *Proc. IEEE Bangalore Humanitarian Technology Conf. (B-HTC)*, Oct. 2020, doi: 10.1109/B-HTC50970.2020.9297839.

[3] A. Lokman, R. K. Ramasamy, and C.-Y. Ting, "Scheduling and predictive maintenance for smart toilet," *IEEE Access*, vol. 11, pp. 17983–17999, 2023, doi: 10.1109/ACCESS.2023.3241942.

[4] T. Stephan, S. M. Hari Krishna, C.-C. Lin, U. Sumesh, S. Agarwal, and H. Kim, "ProWaste for proactive urban waste management using IoT and machine learning," *Sci. Rep.*, vol. 15, Jul. 2025, doi: 10.1038/s41598-025-08452-x.

[5] K. Pardini, J. J. P. C. Rodrigues, O. Diallo, A. K. Das, V. H. C. de Albuquerque, and S. A. Kozlov, "A smart waste management solution geared towards citizens," *Sensors*, vol. 20, no. 8, Art. no. 2380, Apr. 2020, doi: 10.3390/s20082380.

[6] M. Vibhute, J. Biju, K. Shelar, N. Koli, and O. Gubyad, "Smart drainage monitoring system," *Int. J. Multidiscip. Res.*, vol. 7, no. 2, Apr. 2025, doi: 10.36948/ijfmr.2025.v07i02.40325.

[7] Q. P. Ha, S. Metia, and M. D. Phung, "Sensing data fusion for enhanced indoor air quality monitoring," *IEEE Sensors J.*, vol. 20, no. 8, pp. 4430–4441, Apr. 2020, doi: 10.1109/JSEN.2020.2964396.

[8] H. Zhang, R. Zhang, and J. Sun, "Developing real-time IoT-based public safety alert and emergency response systems," *Sci. Rep.*, vol. 15, Aug. 2025, doi: 10.1038/s41598-025-13465-7.

[9] K. Dineva and T. Atanasova, "Design of scalable IoT architecture based on AWS for smart livestock," *Animals*, vol. 11, no. 9, Art. no. 2697, Sep. 2021, doi: 10.3390/ani11092697.

[10] M. Romano, P. Díaz, and I. Aedo, "Gamification-less: May gamification really foster civic participation? A controlled field experiment," *J. Ambient Intell. Humaniz. Comput.*, Jun. 2021, doi: 10.1007/s12652-021-03322-6.